

LECTURE 4

Database Architecture and Data Independence

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

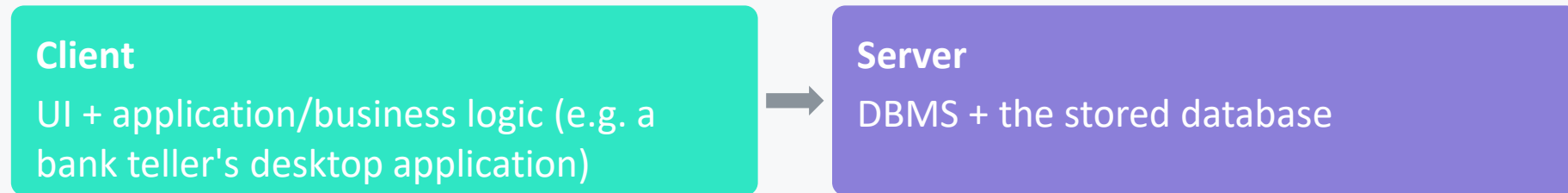
In This Lecture

- Two-tier client-server architecture, and its limitations
- Three-tier client-server architecture, and why it emerged
- The ANSI-SPARC three-schema architecture: external, conceptual, and internal levels
- External/conceptual and conceptual/internal mappings, and what each one does
- Logical vs. physical data independence — precisely defined, with worked examples

Two-Tier Client-Server Architecture

Before looking inside a DBMS, it's worth asking where it physically runs. In a two-tier architecture, the system splits into exactly two parts:

Two-tier client-server architecture



A classic example: an old-style desktop banking application on each teller's workstation, connecting straight to a central database server over the bank's internal network.

The limits of two tiers. This works well for a limited number of clients on a trusted internal network, but every client machine needs the full application installed and updated, business logic is duplicated across every client, and the database server must handle a direct connection from every single client — it stops scaling gracefully as users grow.

Two-Tier, Step by Step: One Teller, One Withdrawal

- The teller enters a withdrawal of 5,000 for account SL21 in the desktop application
- The client builds the SQL statement and sends it straight across the bank's network to the database server
- The DBMS on the server parses, checks, and executes the statement against the stored Account data
- Only the result travels back — the new balance, or an error message
- The client formats that result for the teller's screen; the business logic ran on the teller's own machine

Why Two Tiers Stop Scaling: A Worked Count

- Suppose the bank has 500 tellers, each with the desktop application installed
- Every teller holds an open connection straight to the central database server: 500 direct connections
- A new daily transfer limit is introduced; the rule is copied into 500 separate client installs
- Every one of those installs must be updated and tested — the same rule now lives in 500 places
- Growing to 5,000 users multiplies each of these costs, and the server's connection load with them

Three-Tier Client-Server Architecture

A three-tier architecture splits the client's responsibilities further, inserting a middle layer between the user-facing client and the database server:

Three-tier client-server architecture



A modern online banking website is a textbook three-tier system: the webpage is the presentation tier, the bank's application servers enforcing daily transfer limits are the application tier, and the account database is the data tier.

Client-server architecture is about physical location; the three-schema architecture next is about logical organization. A single-tier desktop database and a three-tier web application can both internally follow the same three-schema organization.

Three-Tier, Step by Step: A Transfer Request

- The customer clicks Transfer in the browser; the presentation tier only shows the form
- The request reaches the application server, which checks the daily transfer limit and the available balance
- Only if the rules pass does the application server ask the database server to update the two accounts
- The database server applies both changes in one transaction and returns a confirmation
- The application server sends the confirmation back, and the browser displays it
- The database never faces the public internet — only trusted application servers talk to it

The ANSI-SPARC Three-Schema Architecture

The standard blueprint for how a DBMS organizes a database's structure into three levels, each describing the same underlying data from a different point of view:

The ANSI-SPARC three-schema architecture

External Level

Many external schemas, one per user group — each shows only the data that group needs

Conceptual Level

One conceptual schema — the complete community view of the whole database's structure and rules

Internal Level

One internal schema — how the data is actually stored on physical media

Three schemas, not three copies of the data. There is exactly one physical copy of the data, described at the internal level. The external and conceptual levels are descriptions — schemas — not additional data stores.

The Three Levels, One Student Record

- Data tier: one physical copy of the record sits in a B-tree-indexed file ordered by studentId
- Conceptual level: the same record is a Student tuple with studentId, name, cnic, and program, plus its business rules
- External level, Course Advisor: the same record appears only as StudentName and Grade for one course
- The advisor's view never shows cnic — yet every value on screen still comes from that one stored record
- Changing the record changes the one copy; each level simply describes that copy differently

External Level

The external level is closest to individual users: a number of external schemas (user views), one per distinct user group, each describing only the part of the database that group cares about.

Worked example: a university enrollment database. The conceptual schema holds full Student, Course, Enrollment, and Instructor relations, including sensitive fields like Student.cnic and Instructor.salary.

Two external schemas over the same conceptual data

External schema: Course Advisor

StudentName, CourseTitle, Grade — no CNIC, no salary data at all

External schema: Payroll Clerk

InstructorName, Department, Salary — no student data at all

Neither view shows the whole conceptual schema, and neither is aware the other exists — each is tailored to exactly what its user group is authorized to see and needs to work with.

Why Not Just Give Everyone the Full Schema?

- Security: the Course Advisor should never see Student.cnic or Instructor.salary
- Simplicity: a view shows only the fields one group needs, reshaped for its task
- Stability: an external schema built for the advisor keeps working when the conceptual schema grows elsewhere
- Privacy: restricting a view is one rule, far more reliable than asking every program to hide fields
- The three-schema design makes these protections the default, not an extra step someone must remember

Conceptual Level and Internal Level

C

Conceptual Level

Exactly one conceptual schema: a single, complete, community-wide view of the entire database's logical structure — every entity, relationship, and constraint — described without reference to physical storage, and without favoring any one external view.

For the university example: the full set of relations (Student, Course, Enrollment, Instructor) with all attributes, primary and foreign keys, and business rules — independent of whether data ends up on a hard disk, an SSD, or split across several servers.

I

Internal Level

Exactly one internal schema, describing the physical storage representation: what files exist, what indexes exist, how records are laid out on disk, what compression or encoding is used. The only level that concerns itself with actual storage media.

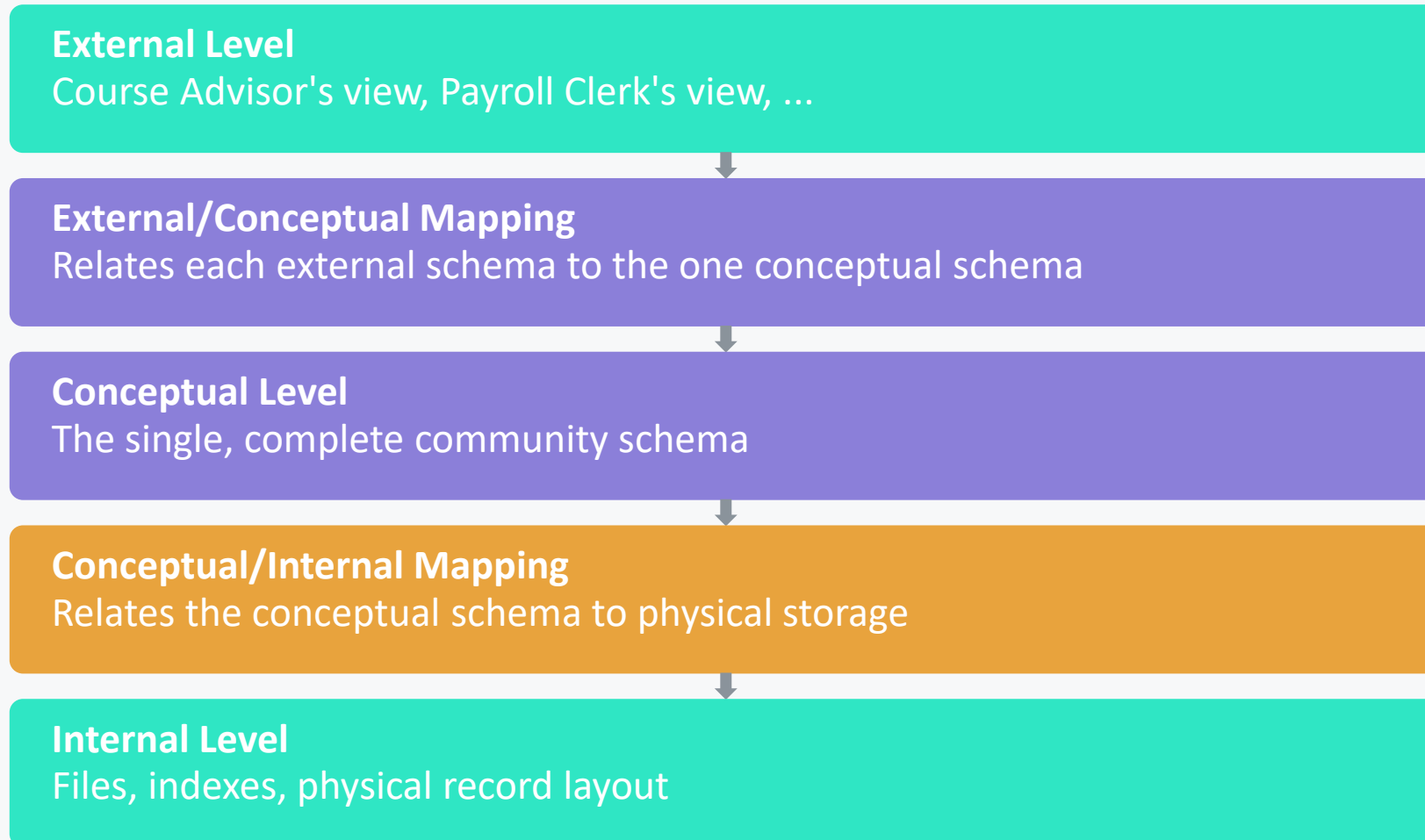
For the university example: Student records stored in a B-tree-indexed file ordered by studentId, with a separate hash index on cnic — details no external-level user, and not even the conceptual schema, needs to know about.

Following One Request Through the Three Levels

- The Course Advisor's report asks for StudentName for student 2021-CS-14
- External/conceptual mapping: StudentName is defined to mean Student.name in the conceptual schema
- Conceptual level: the DBMS identifies the Student tuple whose studentId is 2021-CS-14
- Conceptual/internal mapping: the DBMS finds where that tuple is stored, via the B-tree on studentId
- The stored name is read and returned, reshaped to the advisor's field name
- Each translation handled only its own boundary; no level needed the others' internal details

Mappings Between the Levels

Because the three levels describe the same data differently, the DBMS needs a way to translate a request made at one level into an action at another:



Worked Example: Renaming a Field Without Breaking Anyone

- The conceptual schema renames Student.name to Student.fullName for clarity
- The external/conceptual mapping is updated: StudentName now maps to Student.fullName
- The Course Advisor's external schema is untouched — it still says StudentName
- Reports and applications built against that view keep running with no code change
- Without this mapping layer, every program that referred to name would have to be found and rewritten

Logical Data Independence

The capacity to change the conceptual schema without having to change existing external schemas or the application programs built against them.

It is achieved because a change to the conceptual schema only requires updating the external/conceptual mapping — the external schemas themselves stay exactly as they were.

Example. The university adds a new Instructor.officePhone attribute to the conceptual schema. The Course Advisor's external schema never referenced instructor data at all, so nothing about it needs to change, and every application built against it keeps working — completely unaware the conceptual schema grew a new column.

If the change is “we added a column” or “we split a relation in two,” think logical.

Logical Independence in Practice: Adding and Splitting

- Adding Instructor.officePhone is safe — no external schema referenced instructor contact details
- Splitting Student into Student and Address changes the conceptual schema's shape
- The external/conceptual mapping is rewritten to rebuild the old Student view from the two new relations
- The Course Advisor's external schema still shows the same columns, so its applications keep working
- Limit: if a view truly depends on a removed attribute, no mapping can save it — that view must change

Physical Data Independence

The capacity to change the internal schema — how data is physically stored — without having to change the conceptual schema (and therefore not external schemas or applications either).

Example. The DBA adds a new index on Student.cnic, or migrates storage from spinning disks to SSDs, or reorganizes how Student records are laid out in their file. None of this touches the conceptual schema — only the conceptual/internal mapping changes.

Logical data independence

Conceptual schema changes -> only the external/conceptual mapping updates -> external schemas untouched

Physical data independence

Internal schema changes -> only the conceptual/internal mapping updates -> conceptual schema untouched

Anchor them by which schema changes: logical insulates users from conceptual-level changes; physical insulates the conceptual level from internal, physical-storage changes.

Physical Independence in Practice: Adding an Index

- The DBA adds an index on Student.cnic to speed up lookups by national ID
- The conceptual schema is unchanged: Student still has the same attributes and keys
- Only the conceptual/internal mapping changes, to record that cnic is now indexed
- Every query written against Student runs unchanged; only its execution plan gets faster
- Moving the files from spinning disks to SSDs follows the same pattern — no application code changes

Logical or Physical? Classify the Change

- Add officePhone to Instructor → logical: the conceptual schema changed
- Add an index on Student.cnic → physical: the internal schema changed
- Split Student into Student and Address → logical: the conceptual schema changed
- Move the database files to SSD storage → physical: the internal schema changed
- Rename a column label in one user's report only → neither: it is a mapping inside the external level
- Anchor: ask which schema changed — conceptual points to logical, internal points to physical

From Architecture to Independence: Recap

- Client-server architecture describes where components run: two tiers put logic on every client, three tiers centralize it
- The three-schema architecture describes how one copy of the data is seen at three levels
- Mappings translate each request between levels, and they are the only place that changes when a schema changes
- Logical independence shields users from conceptual changes; physical independence shields the conceptual level from storage changes
- This is the direct answer to Lecture 2's data-dependence problem: changes stay contained instead of cascading

Key Takeaways

- Two-tier client-server architecture splits a system into client and database server; three-tier adds a middle application-server tier for business logic and scalability
- The ANSI-SPARC three-schema architecture organizes a database into external (many user views), conceptual (one complete community schema), and internal (physical storage) levels — three descriptions, not three copies of the data
- The external/conceptual mapping connects user views to the conceptual schema; the conceptual/internal mapping connects the conceptual schema to physical storage
- Logical data independence: changing the conceptual schema without forcing changes to external schemas or applications
- Physical data independence: changing the internal (physical storage) schema without forcing changes to the conceptual schema — and therefore not external schemas either

Looking Ahead: Lecture 5 — The Relational Model

- Lecture 5 formalizes the conceptual level introduced here as the relational model
- You'll meet relations, tuples, attributes, and domains with precise definitions
- You'll learn keys and integrity constraints — the rules the conceptual schema must enforce
- Everything so far in this course — the database approach, languages, and architecture — rests on that model